

CEG 4750/6750 Information Security

Lab/Project 1
Using DES Algorithm

A project submitted in partial fulfillment of the requirements for the
CEG 4750/6750 Information Security

By

Bhuwan Pandey (U01155118)

Bipin Poudel (U01151906)

2026

WRIGHT STATE UNIVERSITY

Introduction

In this project, the DES encryption/decryption programs are provided, and the students (we) will compile and run these programs to encrypt/decrypt files on *fry.cs.wright.edu*. The DES encryption/decryption programs used an existing crypto library, Crypto++ (C++), which was installed on *fry.cs.wright.edu*.

For the project, the very first thing we did was to connect to *fry.cs.wright.edu* and access the shell to do the work. To avoid custom mess and/or inconsistency, we copied the provided files along with directory as it is – to the server and began working. And by "as it is", we mean extracted version of the directory, not the compressed file itself.

After copying the directory to the root of our profile in the server, the *ls* command showed the following (Figure 1.1). Now, since all the necessary files are under the directory *Project 1 attached files Mar 8, 2026 1015 AM*, we had to change current working directory to that directory.

```
[w418bxbp@login01 ~]$ ls
'Project 1 attached files Mar 8, 2026 1015 AM'
[w418bxbp@login01 ~]$ cd Project\ 1\ attached\ files\ Mar\ 8\,\ 2026\ 1015\ AM/
[w418bxbp@login01 Project 1 attached files Mar 8, 2026 1015 AM]$ ls
```

Figure 1.1 – Current working directory

As per the instruction provided and for our own ease, we created an alias called *cryptog++* for the command as shown in the Figure 1.3 below. This helped us avoiding typing such a long command each time we try to compile our program.

```
[w418bxbp@login01 Project 1 attached files Mar 8, 2026 1015 AM]$ alias cryptog++='srtn singularity exec /home/containers/cryptopp.sif g++'
[w418bxbp@login01 Project 1 attached files Mar 8, 2026 1015 AM]$ cryptog++ des_encode_SP2026.cpp -o des_encode -lcryptopp
srtn: job 120767 queued and waiting for resources
srtn: job 120767 has been allocated resources
[w418bxbp@login01 Project 1 attached files Mar 8, 2026 1015 AM]$ cryptog++ des_decode_SP2026.cpp -o des_decode -lcryptopp
srtn: job 120768 queued and waiting for resources
srtn: job 120768 has been allocated resources
[w418bxbp@login01 Project 1 attached files Mar 8, 2026 1015 AM]$
```

Figure 1.2 – Alias and compilation

Also as demonstrated in the above Figure 1.2, we can see that the alias is working perfectly as the compilation of both the encryption/decryption program works. Now, we begin along with the task that we are supposed to complete.

Task 1

For Task 1, we are required to compile and execute the provided source DES encryption/decryption programs, and verify the decrypted file is identical to the original plaintext file. There are total of 5 test files provided to us for the task, which are *MSG1*, *MSG2*, *MSG3*, *text1*, *hw2.pdf.gz*. And for each file, we are required to do the following:

1. Encrypt the file to output it suffixed with *.e* to indicate it as an encrypted file.
2. Decrypt the file to output it suffixed with *.d* to indicate it as a decrypted file.
3. Compare the bytes of couple of encrypted files.
4. Show the difference between two files (the original and the decrypted) to demonstrate the working nature of overall process.

For the subtasks requested, we began encrypting each of the input files one by one as shown in the Figure 1.3 below.

```
[w4l8bxbp@login01 Project 1 attached files Mar 8, 2026 1015 AM]$ ./des_encode MSG1 MSG1.e
The input file name is MSG1
The output file name is MSG1.e
key: 3132333461626364
cipher text stored in:MSG1.e
[w4l8bxbp@login01 Project 1 attached files Mar 8, 2026 1015 AM]$ ./des_encode MSG2 MSG2.e
The input file name is MSG2
The output file name is MSG2.e
key: 3132333461626364
cipher text stored in:MSG2.e
[w4l8bxbp@login01 Project 1 attached files Mar 8, 2026 1015 AM]$ ./des_encode MSG3 MSG3.e
The input file name is MSG3
The output file name is MSG3.e
key: 3132333461626364
cipher text stored in:MSG3.e
[w4l8bxbp@login01 Project 1 attached files Mar 8, 2026 1015 AM]$ ./des_encode text1 text1.e
The input file name is text1
The output file name is text1.e
key: 3132333461626364
cipher text stored in:text1.e
```

Figure 1.3 – Encryption of input files

Similarly, we tried decrypting the encrypted files as shown in the following Figure 1.4.

```
[w4l8bxbp@login01 Project 1 attached files Mar 8, 2026 1015 AM]$ ./des_decode MSG1.e MSG1.d
The input file name is MSG1.e
The output file name is MSG1.d
key: 3132333461626364
recovered text: The most serious incident in NASA since 1986.

plain text stored in:MSG1.d
[w4l8bxbp@login01 Project 1 attached files Mar 8, 2026 1015 AM]$ ./des_decode MSG2.e MSG2.d
The input file name is MSG2.e
The output file name is MSG2.d
key: 3132333461626364
recovered text: The most serious accident in NASA since 1986.

plain text stored in:MSG2.d
[w4l8bxbp@login01 Project 1 attached files Mar 8, 2026 1015 AM]$ ./des_decode MSG3.e MSG3.d
The input file name is MSG3.e
The output file name is MSG3.d
key: 3132333461626364
recovered text: The most serious incident in NASA since 1985.

plain text stored in:MSG3.d
```

Figure 1.4 – Decryption of encrypted files

Task 1.1

So far, so good. For the subtask 1.1, we print the octal byte values using the command `od -b filename` for each file as shown in the below Figure 1.1.1.

```
[w4l8bxbp@login01 Project 1 attached files Mar 8, 2026 10:15 AM]$ od -b MSG1.e
0000000 341 145 130 351 270 360 041 227 026 153 311 211 317 140 023 262
0000020 010 341 060 376 123 265 357 215 141 062 014 226 107 143 073 114
0000040 025 273 114 373 107 151 221 356 137 171 310 340 106 266 113 173
0000060
[w4l8bxbp@login01 Project 1 attached files Mar 8, 2026 10:15 AM]$ od -b MSG2.e
0000000 341 145 130 351 270 360 041 227 026 153 311 211 317 140 023 262
0000020 203 174 120 003 341 317 233 361 141 062 014 226 107 143 073 114
0000040 025 273 114 373 107 151 221 356 137 171 310 340 106 266 113 173
0000060
[w4l8bxbp@login01 Project 1 attached files Mar 8, 2026 10:15 AM]$ od -b MSG1.e
0000000 341 145 130 351 270 360 041 227 026 153 311 211 317 140 023 262
0000020 010 341 060 376 123 265 357 215 141 062 014 226 107 143 073 114
0000040 025 273 114 373 107 151 221 356 137 171 310 340 106 266 113 173
0000060
[w4l8bxbp@login01 Project 1 attached files Mar 8, 2026 10:15 AM]$ od -b MSG3.e
0000000 341 145 130 351 270 360 041 227 026 153 311 211 317 140 023 262
0000020 010 341 060 376 123 265 357 215 141 062 014 226 107 143 073 114
0000040 025 273 114 373 107 151 221 356 055 107 163 260 125 255 041 270
0000060
```

Figure 1.1.1 – Octal byte values of encrypted files

Task 1.2

From the above Figure 1.1.1, we can conclude for the comparison of *MSG1.e* and *MSG2.e* that there are total of 8 bytes of difference as in the second row – the byte difference starts from the first segment to all the way up to 8th segment.

Similarly, for the *MSG1.e* and *MSG3.e*, the third row indicates that there are total of 8 bytes difference starting from 9th segment to all the way up to 16th.

This overall difference is actually expected with DES as its block size is 8 bytes and changing one block affects exactly one ciphertext block.

Task 1.3

We are required to use *diff* command to show the difference of two files (the original file, and the decrypted file). The result is shown in the Figure 1.3.1 below:

```
[w4l8bxbp@login01 Project 1 attached files Mar 8, 2026 10:15 AM]$ diff MSG1 MSG1.d
[w4l8bxbp@login01 Project 1 attached files Mar 8, 2026 10:15 AM]$ diff MSG2 MSG2.d
[w4l8bxbp@login01 Project 1 attached files Mar 8, 2026 10:15 AM]$ diff MSG3 MSG3.d
[w4l8bxbp@login01 Project 1 attached files Mar 8, 2026 10:15 AM]$ diff text1 text1.d
[w4l8bxbp@login01 Project 1 attached files Mar 8, 2026 10:15 AM]$
```

Figure 1.3.1 – Difference between files

As we can observe from the above Figure 1.3.1, we can clearly conclude that the encryption and decryption works perfectly as there are no differences shown for all the comparisons made.

Task 2

For this task, we are provided with the source DES encryption/decryption programs that performs in ECB mode. We are required to change the source code files of the DES programs, so that the programs are working in CBC mode while still verifying that the decrypted file is identical to its original plaintext file.

The below Figure 1.2.1 shows the modification made in the function *des_encode* and *main*. We have added a parameter of *byte iv[]* and tweaked the DES to operate on *CBC_Mode*. The very similar logic is applied to the decryption program as shown in the Figure 1.2.2. Both the file's name are *des_cbc_encode_SP2026.cpp* and *des_cbc_decode_SP2026.cpp*. These files are saved as separate ones and renamed to isolate each programs from the other.

```
string des_encode(string & plain, byte key[], byte iv[])
{
    string cipher;
    try
    {
        //cout << "plain text: " << plain << endl;
        CBC_Mode<DES>::Encryption enc;
        enc.SetKeyWithIV(key, DES::DEFAULT_KEYLENGTH, iv);
        StringSource(plain, true, new StreamTransformationFilter(enc, new StringSink(cipher)));
    }
    catch(const CryptoPP::Exception& e)
    {
        cerr << e.what() << endl;
        exit(1);
    }
    return cipher;
}

int main(int argc, char * argv[])
{
    ifstream file1;
    ofstream file2;
    byte key[DES::DEFAULT_KEYLENGTH]={'1','2','3','4','a','b','c','d'}; //key is hardcoded
    byte iv[DES::BLOCKSIZE] = {0}; // 8 byte iv (all zeros for just for test's sake)
```

Figure 1.2.1 – Encryption program in CBC mode

```

string des_decode(string & cipher,byte key[], byte iv[])
{
    string plain;
    //decryption
    try
    {
        CBC_Mode< DES >::Decryption dec;
        dec.SetKeyWithIV(key, DES::DEFAULT_KEYLENGTH,iv);
        StringSource s(cipher, true, new StreamTransformationFilter(dec, new StringSink(plain)));
        cout << "recovered text: " << plain<< endl;
    }
    catch(const CryptoPP::Exception& e)
    {
        cerr << e.what() << endl;
        exit(1);
    }
    return plain;
}

int main(int argc,char * argv[])
{
    ifstream file1;
    ofstream file2;

    byte key[DES::DEFAULT_KEYLENGTH]={'1','2','3','4','a','b','c','d'}; //key is hardcoded;
    byte iv[DES::BLOCKSIZE] = {0}; // the same iv used in encryption
}

```

Figure 1.2.2 – Decryption program in CBC mode

As we have noticed already, the CBC mode requires an initialization vector (IV) to encrypt and decrypt a file where the size of the IV is set to the DES's default length. i.e. 8 bytes typically. The below Figure 1.2.3 shows the compilation of the two programs on CBC mode, the Figure 1.2.4 shows the encryption of the files, the Figure 1.2.5 shows the decryption as well as the comparison between them.

```

[w4l8bxb@login01 Project 1 attached files Mar 8, 2026 1015 AM]$ cryptog++ des_cbc_encode_SP2026.cpp -lcryptopp -o des_cbc_encode
srn: job 120614 queued and waiting for resources
srn: job 120614 has been allocated resources
[w4l8bxb@login01 Project 1 attached files Mar 8, 2026 1015 AM]$ cryptog++ des_cbc_decode_SP2026.cpp -lcryptopp -o des_cbc_decode
srn: job 120615 queued and waiting for resources
srn: job 120615 has been allocated resources

```

Figure 1.2.3 – Compilation of programs on CBC mode

```

[w418bxbp@login01 Project 1 attached files Mar 8, 2026 1015 AM]$ ./des_cbc_encode MSG1 MSG1.cbc.e
The input file name is MSG1
The output file name is MSG1.cbc.e
key: 3132333461626364
cipher text stored in:MSG1.cbc.e
[w418bxbp@login01 Project 1 attached files Mar 8, 2026 1015 AM]$ ./des_cbc_encode MSG2 MSG2.cbc.e
The input file name is MSG2
The output file name is MSG2.cbc.e
key: 3132333461626364
cipher text stored in:MSG2.cbc.e
[w418bxbp@login01 Project 1 attached files Mar 8, 2026 1015 AM]$ ./des_cbc_encode MSG3 MSG3.cbc.e
The input file name is MSG3
The output file name is MSG3.cbc.e
key: 3132333461626364
cipher text stored in:MSG3.cbc.e
[w418bxbp@login01 Project 1 attached files Mar 8, 2026 1015 AM]$ ./des_cbc_encode text1 text1.cbc.e
The input file name is text1
The output file name is text1.cbc.e
key: 3132333461626364
cipher text stored in:text1.cbc.e
[w418bxbp@login01 Project 1 attached files Mar 8, 2026 1015 AM]$

```

Figure 1.2.4 – Encryption on CBC mode

```

[w418bxbp@login01 Project 1 attached files Mar 8, 2026 1015 AM]$ ./des_cbc_decode MSG3.cbc.e MSG3.cbc.d
The input file name is MSG3.cbc.e
The output file name is MSG3.cbc.d
key: 3132333461626364
recovered text: The most serious incident in NASA since 1985.

plain text stored in:MSG3.cbc.d
[w418bxbp@login01 Project 1 attached files Mar 8, 2026 1015 AM]$ ./des_cbc_decode MSG2.cbc.e MSG2.cbc.d
The input file name is MSG2.cbc.e
The output file name is MSG2.cbc.d
key: 3132333461626364
recovered text: The most serious accident in NASA since 1986.

plain text stored in:MSG2.cbc.d
[w418bxbp@login01 Project 1 attached files Mar 8, 2026 1015 AM]$ ./des_cbc_decode MSG1.cbc.e MSG1.cbc.d
The input file name is MSG1.cbc.e
The output file name is MSG1.cbc.d
key: 3132333461626364
recovered text: The most serious incident in NASA since 1986.

plain text stored in:MSG1.cbc.d
[w418bxbp@login01 Project 1 attached files Mar 8, 2026 1015 AM]$ diff MSG1 MSG1.cbc.d
[w418bxbp@login01 Project 1 attached files Mar 8, 2026 1015 AM]$ diff text1 text1.cbc.d
[w418bxbp@login01 Project 1 attached files Mar 8, 2026 1015 AM]$

```

Figure 1.2.5 – Decryption on CBC mode

Task 3

This task requires us to change the source code files of the DES encryption/decryption programs which has its key hardcoded. The changes needs to be such that the key would be entered through the command line as the 4th input argument. The below Figure 1.3.1 and Figure 1.3.2 shows the tiny changes made in encryption and decryption program respectively. The file's name are *des_KEY_PARAM_encode_SP2026.cpp* and *des_KEY_PARAM_decode_SP2026.cpp* respectively for the encryption and decryption program.

```

int main(int argc, char * argv[])
{
    ifstream file1;
    ofstream file2;
    byte key[DES::DEFAULT_KEYLENGTH];

    string keyInput = argv[3];
    if(keyInput.length() != DES::DEFAULT_KEYLENGTH)
    {
        cout << "Key must be 8 characters long !!!" << endl;
        exit(1);
    }
    memcpy(key, keyInput.data(), DES::DEFAULT_KEYLENGTH);
}

```

Figure 1.3.1 – Encryption program with key parameter

```

int main(int argc, char * argv[])
{
    ifstream file1;
    ofstream file2;

    byte key[DES::DEFAULT_KEYLENGTH];
    byte iv[DES::BLOCKSIZE] = {0}; // the same iv used in encryption

    if(argc!=4)
    {
        cout<<"usage:des_decode infile outfile" <<endl;
    }

    string keyInput = argv[3];
    if(keyInput.length() != DES::DEFAULT_KEYLENGTH)
    {
        cout << "Key must be exactly 8 characters !!!" << endl;
        exit(1);
    }

    memcpy(key, keyInput.data(), DES::DEFAULT_KEYLENGTH);
}

```

Figure 1.3.2 – Decryption program with key parameter

If we observe carefully on both the programs, we can see that we have removed hardcoded array of keys and are using the third (starting with 0) parameter as a key instead. It is to be noted that the length of the key should be the algorithm's default segment/block size. i.e. 8. The following Figure 1.3.3 and Figure 1.3.4 shows the successful encryption and decryption of the files respectively.

```
[w4l8bxbp@login01 Project 1 attached files Mar 8, 2026 10:15 AM]$ ./des_KEY_PARAM_encode MSG3 MSG3_KEY_PARAM.e 1234abcd
The input file name is MSG3
The output file name is MSG3_KEY_PARAM.e
key: 3132333461626364
cipher text stored in:MSG3_KEY_PARAM.e
[w4l8bxbp@login01 Project 1 attached files Mar 8, 2026 10:15 AM]$ ./des_KEY_PARAM_encode MSG4 MSG4_KEY_PARAM.e 1234abcd
The input file name is MSG4
The output file name is MSG4_KEY_PARAM.e
key: 3132333461626364
cipher text stored in:MSG4_KEY_PARAM.e
[w4l8bxbp@login01 Project 1 attached files Mar 8, 2026 10:15 AM]$ ./des_KEY_PARAM_encode text1 text1_KEY_PARAM.e 1234abcd
The input file name is text1
The output file name is text1_KEY_PARAM.e
key: 3132333461626364
cipher text stored in:text1_KEY_PARAM.e
```

Figure 1.3.3 – Encryption of files with key parameter program (encryptor)

```
[w4l8bxbp@login01 Project 1 attached files Mar 8, 2026 10:15 AM]$ ./des_KEY_PARAM_decode MSG3_KEY_PARAM.e MSG3_KEY_PARAM.d 1234abcd
The input file name is MSG3_KEY_PARAM.e
The output file name is MSG3_KEY_PARAM.d
key: 3132333461626364
recovered text: The most serious incident in NASA since 1985.

plain text stored in:MSG3_KEY_PARAM.d
[w4l8bxbp@login01 Project 1 attached files Mar 8, 2026 10:15 AM]$ ./des_KEY_PARAM_decode MSG2_KEY_PARAM.e MSG2_KEY_PARAM.d 1234abcd
The input file name is MSG2_KEY_PARAM.e
The output file name is MSG2_KEY_PARAM.d
key: 3132333461626364
recovered text: The most serious accident in NASA since 1986.

plain text stored in:MSG2_KEY_PARAM.d
[w4l8bxbp@login01 Project 1 attached files Mar 8, 2026 10:15 AM]$ ./des_KEY_PARAM_decode MSG1_KEY_PARAM.e MSG1_KEY_PARAM.d 1234abcd
The input file name is MSG1_KEY_PARAM.e
The output file name is MSG1_KEY_PARAM.d
key: 3132333461626364
recovered text: The most serious incident in NASA since 1986.

plain text stored in:MSG1_KEY_PARAM.d
```

Figure 1.3.4 – Decryption of files with key parameter program (decryptor)

While the program seemed to run perfectly fine, we still needed to check the difference between the original file and its decrypted equivalent just to make sure that integrity is not lost. The below Figure 1.3.5 shows the difference between the files *MSG1* and *text1*, and the octal bytes value resulting similar output as it did in previous tests. To save time and effort, we assume that if the program succeeds on encrypting those two files, it succeeds on any.

```
[w4l8bxbp@login01 Project 1 attached files Mar 8, 2026 10:15 AM]$ diff MSG1 MSG1_KEY_PARAM.d
[w4l8bxbp@login01 Project 1 attached files Mar 8, 2026 10:15 AM]$ diff text1 text1_KEY_PARAM.d
[w4l8bxbp@login01 Project 1 attached files Mar 8, 2026 10:15 AM]$ od -b MSG1_KEY_PARAM.e
00000000 341 145 130 351 270 360 041 227 252 075 352 303 210 260 211 315
00000020 336 003 273 355 337 325 360 240 125 320 344 131 115 026 251 252
00000040 035 223 214 231 334 031 154 357 141 306 005 002 314 145 052 326
00000060
[w4l8bxbp@login01 Project 1 attached files Mar 8, 2026 10:15 AM]$ od -b MSG2_KEY_PARAM.e
00000000 341 145 130 351 270 360 041 227 252 075 352 303 210 260 211 315
00000020 124 041 334 247 343 005 113 113 043 166 244 166 076 205 164 074
00000040 367 044 050 322 265 347 021 304 224 273 217 074 116 267 304 033
00000060
[w4l8bxbp@login01 Project 1 attached files Mar 8, 2026 10:15 AM]$ od -b MSG3_KEY_PARAM.e
00000000 341 145 130 351 270 360 041 227 252 075 352 303 210 260 211 315
00000020 336 003 273 355 337 325 360 240 125 320 344 131 115 026 251 252
00000040 035 223 214 231 334 031 154 357 004 353 045 217 141 352 006 201
00000060
[w4l8bxbp@login01 Project 1 attached files Mar 8, 2026 10:15 AM]$
```

Figure 1.3.5 – Difference between files

Conclusion

This project provided practical experience with symmetric encryption by implementing and modifying DES encryption and decryption programs using the Crypto++ library in a Linux server environment. By compiling and executing the provided programs on *fry.cs.wright.edu*, we verified the correctness of the encryption/decryption process by confirming that the decrypted files were identical to the original plaintext files. Through the comparison of ciphertext outputs using the *od* command, we observed how small differences in plaintext propagate to changes in ciphertext bytes. Additionally, modifying the implementation from ECB mode to CBC mode helped illustrate the importance of block cipher modes in improving cryptographic security.

Finally, allowing the encryption key to be supplied through command line arguments improved the flexibility and usability of the program. Overall, the project strengthened understanding of DES operations, block cipher modes, and practical use of cryptographic libraries in real computing environments.